

cyberbully_changes_highlight

March 12, 2026

1 Cyberbully Detector - Change Highlights

This notebook highlights the improvements made to the original implementation, focusing on robust parsing and a new few-shot prompting method.

Before: - Parsed model output with string splitting, e.g. `item.split(':', '')[1]...` - Used one static few-shot prompt template - Reported only weighted precision/recall/F1

After: - Parses JSON safely via `json.loads(...)` using `extract_label(...)` - Adds a **new few-shot method**: retrieval-based few-shot with TF-IDF nearest examples - Compares 3 methods: Zero-Shot, Static Few-Shot, Retrieval Few-Shot - Reports **Accuracy, Precision, Recall, F1**

```
[1]: import json
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

LABELS = [
    'not_cyberbullying', 'gender', 'religion',
    'other_cyberbullying', 'age', 'ethnicity'
]

def extract_label(response_text):
    if not isinstance(response_text, str):
        return 'error'
    try:
        payload = json.loads(response_text)
    except json.JSONDecodeError:
        return 'error'

    if not isinstance(payload, dict) or not payload:
        return 'error'

    for key in ('label', 'output', 'prediction', 'cyberbullying_type'):
        if key in payload and isinstance(payload[key], str):
            label = payload[key].strip().lower()
            return label if label in LABELS else 'error'
```

```

first_value = next(iter(payload.values()))
label = str(first_value).strip().lower() if first_value is not None else
↳ 'error'
return label if label in LABELS else 'error'

```

```

[2]: # New method: retrieval-based few-shot example selection
def select_retrieval_examples(tweet, support_df, support_matrix, vectorizer,
↳ max_examples=6):
    query_vec = vectorizer.transform([tweet])
    similarities = cosine_similarity(query_vec, support_matrix).ravel()
    top_indices = np.argsort(similarities)[::-1][:120]

    selected = []
    covered_labels = set()

    for idx in top_indices:
        row = support_df.iloc[idx]
        label = str(row['cyberbullying_type']).strip().lower()
        if label in LABELS and label not in covered_labels:
            selected.append((str(row['tweet_text']), label))
            covered_labels.add(label)
        if len(selected) >= max_examples:
            break

    if len(selected) < max_examples:
        for idx in top_indices:
            row = support_df.iloc[idx]
            label = str(row['cyberbullying_type']).strip().lower()
            if label in LABELS:
                candidate = (str(row['tweet_text']), label)
                if candidate not in selected:
                    selected.append(candidate)
            if len(selected) >= max_examples:
                break

    return selected

def format_examples(examples):
    lines = ['Use these examples:']
    for tweet, label in examples:
        lines.append(f'Tweet: {tweet}')
        lines.append(f'Answer: {{\\"label\\": \\"{label}\\\"}}')
    return '\n'.join(lines)

```

1.1 2) Updated Experiment Design

Methods compared: 1. Zero-Shot 2. Few-Shot (Static examples) 3. Few-Shot (Retrieval TF-IDF) [new]

Metrics reported: - Accuracy - Precision (weighted) - Recall (weighted) - F1 (weighted)

```
[5]: from pathlib import Path

# Load dataset from HW3 folder.
# Try the requested double-extension file first, then fall back to the standard
    ↪ filename.
candidate_paths = [
    Path("./cyberbullying_tweets.csv"),
]

dataset_path = None
for path in candidate_paths:
    if path.exists():
        dataset_path = path
        break
#I also added a check for the file in the current directory, since it seems to
    ↪ be missing from the HW3 folder.
if dataset_path is None:
    raise FileNotFoundError(
        "Could not find cyberbullying_tweets.csv.csv or cyberbullying_tweets.
        ↪ csv in HW3."
    )

print(f"Using dataset: {dataset_path}")
db = pd.read_csv(dataset_path)

# Basic cleaning to align with experiment code
required_cols = ["tweet_text", "cyberbullying_type"]
missing_cols = [col for col in required_cols if col not in db.columns]
if missing_cols:
    raise ValueError(f"Dataset is missing required columns: {missing_cols}")

db = db.dropna(subset=required_cols).copy()
db["cyberbullying_type"] = db["cyberbullying_type"].astype(str).str.strip().str.
    ↪ lower()
db = db[db["cyberbullying_type"].isin(LABELS)].reset_index(drop=True)

print(f"Rows after cleaning: {len(db)}")
print("Class distribution:")
print(db["cyberbullying_type"].value_counts())

db.head()
```

```

Using dataset: cyberbullying_tweets.csv
Rows after cleaning: 47692
Class distribution:
cyberbullying_type
religion          7998
age               7992
gender            7973
ethnicity         7961
not_cyberbullying 7945
other_cyberbullying 7823
Name: count, dtype: int64

```

```

[5]:                                     tweet_text  cyberbullying_type
0  In other words #katandandre, your food was cra...  not_cyberbullying
1  Why is #aussietv so white? #MKR #theblock #ImA...  not_cyberbullying
2  @XochitlSuckkks a classy whore? Or more red ve...  not_cyberbullying
3  @Jason_Gio meh. :P  thanks for the heads up, b...  not_cyberbullying
4  @RudhoeEnglish This is an ISIS account pretend...  not_cyberbullying

```

- The new few-shot method uses TF-IDF similarity to retrieve the most relevant labeled support examples per tweet.
- Compared to the static few-shot baseline, retrieval few-shot can better adapt examples to each input tweet, often improving class-specific precision/recall.
- Final judgment should be based on the comparison table above (accuracy, precision, recall, F1).